DynaByte: Shared-Budget Allocation of Expert Precision, Residency, and Lookahead

ANONYMOUS AUTHOR(S)

How should a limited GPU allocate its expert memory and host-to-device transfer bandwidth among expert precision, residency, and lookahead, so that Mixture-of-Experts inference reduces tail latency while the expert-memory peak stays within budget? Precision spends device bytes and enlarges a later transfer, residency spends bytes to avoid a transfer, and lookahead spends both bytes and bandwidth to hide a transfer. Policies that fix two of these uses and tune the third can still raise tail latency once the two budgets are shared. DynaByte admits each byte exchange only when the claim it displaces lowers predicted exposed wait and the quality-risk target still holds; one ledger publishes a representation after its copy completes and keeps the expert-memory peak within budget. On Qwen3-30B, under that budget, p95 TPOT is 45.6 ms against 51.8 ms for HOBBIT. The DynaByte interval overlaps a frozen joint assignment, so that comparison is unresolved.

CCS Concepts: • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → **Memory management**; • **Computer systems organization** → *Architectures*.

Additional Key Words and Phrases: mixture of experts, memory management, prefetching, quantization, performance modeling

ACM Reference Format:
Anonymous Author(s). 2027. DynaByte: Shared-Budget Allocation of Expert Precision, Residency, and Lookahead. *Proc. ACM Meas. Anal. Comput. Syst.* 1, 1 (October 2027), 25 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

A Mixture-of-Experts (MoE) layer routes each token to a small subset of expert subnetworks, allowing model capacity to grow without a proportional increase in per-token computation [16, 30, 40]. The memory footprint remains large because every expert representation must still be stored or made available at inference time. Models such as Mixtral, DeepSeek-V2, and Qwen3 contain tens to hundreds of billions of parameters while activating only a fraction of them for each token [9, 24, 50]. GLaM scales the same sparse pattern to a trillion-parameter language model [11]. On a single accelerator, expert weights compete with non-expert parameters, the KV cache, activations, and runtime workspaces for device memory. When experts cannot all remain at the desired precision, the system must both compress them and move them across the host-to-device link.

How should a limited GPU allocate its expert memory and host-to-device transfer bandwidth among expert precision, residency, and lookahead, so that Mixture-of-Experts inference reduces tail latency while the expert-memory peak stays within budget?

These two budgets meet in three uses. Precision spends expert-memory bytes on a higher-precision representation and increases the size of any later transfer. Residency spends bytes to keep an expert on the device so that a later access requires no transfer. Lookahead spends both bytes and bandwidth to stage an absent expert early enough that an otherwise unavoidable transfer overlaps preceding computation. A byte assigned to precision cannot also retain another expert or stage a future one. Host-to-device bandwidth determines whether that early transfer finishes before its deadline.

Prior work has developed effective policies for parts of this problem. Quantization and mixed-precision methods choose expert representations under a memory or accuracy target [13, 18, 32]. Expert caches and prefetchers decide which experts to keep and when to transfer missing experts,

2027. 2476-1249/2027/10-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

commonly under a fixed representation size [15, 41, 43, 49]. Mixed-precision offloading systems demonstrate that lower-precision transfers, caching, and prefetching can coexist [21, 45]. The right assignment still changes with the workload: expert sensitivity sets the value of precision, routing locality sets the value of residency, and transfer deadlines set the value of lookahead. A fixed partition or a fixed priority among the three uses cannot express these changing opportunity costs. These systems show that the three mechanisms can operate together, yet they do not admit precision, residency, and lookahead through one exchange on one byte ledger.

DynaByte answers this question under a hard expert-memory budget B. Admission checks a quality-risk target $\widehat{\epsilon}$; measured ΔQ is reported against ϵ. The decision is a byte exchange: a requested expert state together with the precision, residency, or lookahead claim that funds it. Residency, lookahead, and precision share one admission path and one byte ledger, with no private quota. The executor publishes a representation only after its copy completes, and the expert-memory peak stays within B.

We make three contributions.

- We formulate the joint allocation of expert memory and host-to-device bandwidth among precision, residency, and lookahead. On Qwen3-30B, p95 TPOT is 45.6 ms against 51.8 ms for HOBBIT, with HOBBIT outside DynaByte on all three models, and the DynaByte interval overlaps Static-Joint.
- We develop a byte-lease exchange that names the memory each request displaces. On Qwen3-30B, separate quotas raise p95 TPOT from 458 ms to 524 ms. Dropping the counterfactual replay of that exchange does not separate the tail from DynaByte.
- We design a runtime with one byte ledger and a publish-after-copy protocol. At the implemented point, deadline replay averages 0.15 ms, against about 8.3 ms for one layer, and a transition is published only after its copy completes.

2 BACKGROUND AND MOTIVATION

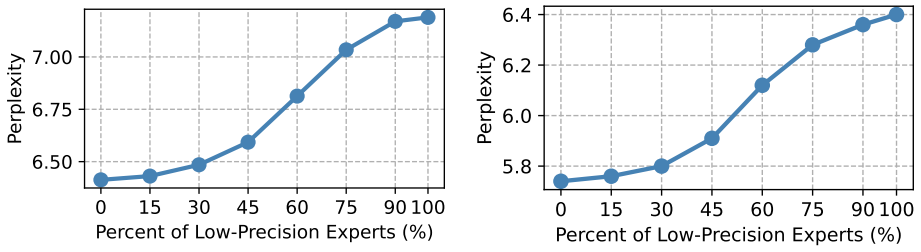
2.1 Expert Memory on One Accelerator

An MoE layer routes each token to k_ℓ experts. The router determines which experts execute. Expert-choice routing lets each expert select its tokens [60]. It does not determine where their weights reside or when they arrive on the device. A selected expert that is absent must be transferred from host memory, and a demand transfer exposes that time on the forward path.

Memory for non-expert parameters, the KV cache, activations, and fixed workspaces is reserved first. The remaining capacity is the expert-memory budget B. Host memory stores a packed low-precision representation and a packed high-precision representation of every routed expert, with aggregate footprints P_{lo} and P_{hi}. We write $\Phi = B/P_{\text{lo}}$ and $\rho = P_{\text{hi}}/P_{\text{lo}}$. At any instant an expert is absent or resident at low or high precision. The forward pass can use only a published resident representation.

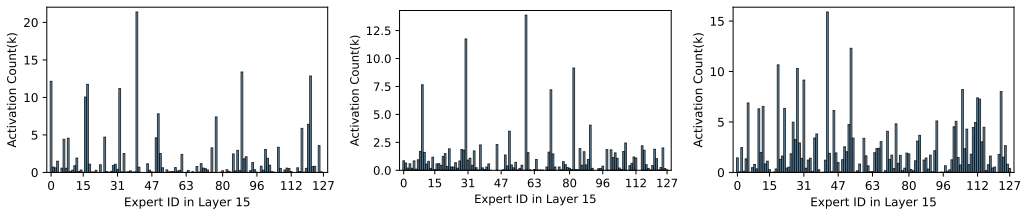
The host-to-device bandwidth budget is the measured copy rate C of the migration queue. C is the rate of that one queue. Representation size, whether the expert stays resident, and when its transfer occupies the queue decide which transfers consume this rate.

The same expert-memory bytes have three uses. The extra bytes of a higher-precision representation are *fidelity*, and the rest of the paper uses fidelity for this use. *Residency* keeps an expert on the device so that a later access requires no transfer. *Lookahead* stages an absent expert early enough for its transfer to overlap preceding computation. Lookahead is a timed transition from absence to residency. It is not a separate representation.



(a) Qwen3-30B: FP16/INT4 (b) Qwen3-Next-80B: INT4/INT2

Fig. 1. WikiText-2 perplexity under frozen precision-quota sweeps. Experts are demoted in the order defined by an independent, fixed ranking; these are not online-policy results.



(a) WikiText (b) GSM8K (c) HumanEval

Fig. 2. Selected token–expert dispatch counts at Layer 15 of Qwen3-30B. All experts remain at low precision and the counters are reset between workloads.

2.2 Fidelity Has a Quality Price and a Transfer Cost

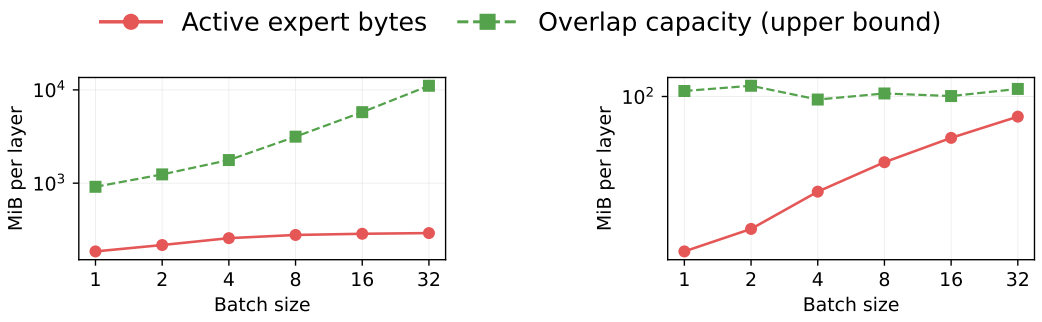
Figure 1 changes the fraction of experts stored at low precision. Each point demotes a longer suffix of one fixed ranking, so the curve prices a fidelity quota. The ranking stays frozen, and the sweep does not reassign which experts receive the higher-precision bytes.

WikiText-2 perplexity rises on both Qwen3-30B (FP16/INT4) and Qwen3-Next-80B (INT4/INT2) as more experts move to the lower tier. The rise is continuous, so a quality-degradation limit selects how large the low-precision fraction may become. The higher-precision representation is also the larger packed copy. Raising the fidelity share therefore spends expert memory and increases the bytes a later transfer must move. Which experts should receive those bytes is still open.

2.3 The Experts a Workload Uses Also Change

The quality curve leaves the recipients unnamed. Figure 2 counts the token–expert dispatches at Layer 15 of Qwen3-30B on WikiText, GSM8K, and HumanEval. Every expert stays at low precision, and the counters reset between workloads, so the histograms show which experts each workload uses.

The three tasks dispatch different experts. A fidelity assignment or a resident set taken from one workload therefore misses experts the next workload uses. The counts record access frequency. They leave quantization sensitivity unmeasured, so they do not rank the quality harm of a low-precision expert. Both the fidelity recipients and the resident set are workload-dependent inputs to the same allocation.



(a) Prefill (b) Decode

Fig. 3. Active expert bytes and an upper bound on overlap capacity for Qwen3-30B. The bound uses measured blocking-copy bandwidth and mean per-layer TTFT or TPOT; it is not the result of an overlapping offload runtime.

2.4 Whether the Remaining Bytes Can Be Hidden Also Changes

The quality curve prices the fidelity quota, and the dispatch counts show that the accessed experts change with the task. What remains open is whether an expert that is still absent can arrive before it is used. Figure 3 compares, for Qwen3-30B, the bytes selected in one MoE layer with an upper bound on the bytes that can move during that layer's compute.

Demand comes from five measured prompt blocks at each batch size. The bound multiplies measured blocking host-to-device bandwidth by mean TTFT or TPOT and divides by 48 routed layers. The bound is an optimistic ceiling: it ignores each expert's deadline and the bandwidth concurrent kernels take. It is the capacity of the measured copy rate, not the result of an overlapping offload runtime.

Prefill touches most routed experts, and its compute window grows with the batch, so the overlap bound rises with demand. Decode touches fewer experts, but its shorter window leaves less time before use. The gap between selected bytes and this bound is the traffic lookahead cannot hide at the measured rate. Prefill and decode move that gap in opposite directions, so one lookahead depth does not fit both phases. Representation size stays fixed in this comparison.

2.5 A Fidelity Change Moves Residency and Lookahead

Figure 4 places fidelity, residency, and lookahead on one expert-memory budget at the measured host-to-device bandwidth of the replay. The experiment replays a measured 2,048-token routing trace under the byte-accurate expert budget used by the Qwen3-30B runtime. An illustrative WikiText-2 limit of 2.1 perplexity above the all-high configuration leaves only the 25% and 40% high-precision fractions feasible among the scanned shares.

Residents follow the frequency ranking from warm-up traces. Lookahead has oracle knowledge of the measured active set, and its staging slots and the measured transfer window limit how many experts it can stage. This replay is not an end-to-end DynaByte result.

At 25% high precision, 12 staging slots leave 96 low-precision residents per layer and reduce exposed transfer to 0.44 ms/request. At 40% high precision, exposed transfer is minimized at eight slots with 43 low-precision residents, then rises because additional staging displaces residents. Raising the high-precision share from 25% to 40% therefore changes both residency and the staging

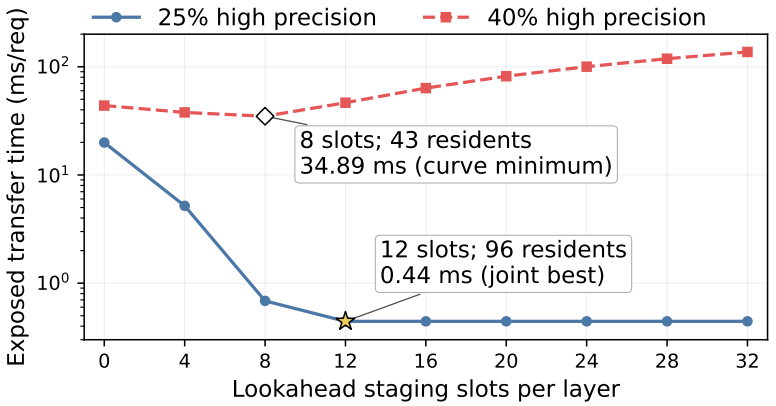


Fig. 4. Trace replay of the two quality- and memory-feasible precision allocations on Qwen3-30B, at the measured host-to-device bandwidth of this trace. Increasing the high-precision share from 25% to 40% reduces residency and moves the best staging allocation from 12 to 8 slots. Bandwidth is not swept. Lookahead uses oracle knowledge of the measured active set; this is not an end-to-end DynaByte result.

depth that reduces exposed transfer. Host-to-device bandwidth stays at the measured rate of this trace.

2.6 Motivation and Design Challenges

Together, the four measurements are one allocation problem. Quantization, caching, and prefetching can coexist in one system. A fixed quota or a fixed priority cannot compare the opportunity cost of fidelity, residency, and lookahead as the workload changes.

Challenge 1: heterogeneous value under a quality-risk target. Fidelity reduces quality risk, residency avoids a future transfer, and lookahead reduces the exposed portion of a transfer that still occurs. These benefits have different units and last for different amounts of time. Admission checks estimated risk, and measured ΔQ is reported against ϵ. Risk is not converted into milliseconds or added to the latency objective.

Challenge 2: future-dependent value at different time scales. Routing locality and deadlines change faster than the quality evidence required to justify a fidelity transition. The same allocation still assigns expert memory and transfer bandwidth, while the evidence and copy cost of fidelity take longer to amortize.

Challenge 3: hard memory safety during transitions. A final resident set that fits in B can still pass through a transition that does not. Promotion and prefetch require destination bytes before a copy begins, and an old published representation remains occupied until its readers finish. A representation still in transfer cannot execute.

3 DESIGN

3.1 Overview

DynaByte answers the allocation question with one admission procedure, shown in Figure 5. The procedure chooses each expert's representation, residency, and transfer issue time, then commits those choices as one exchange.

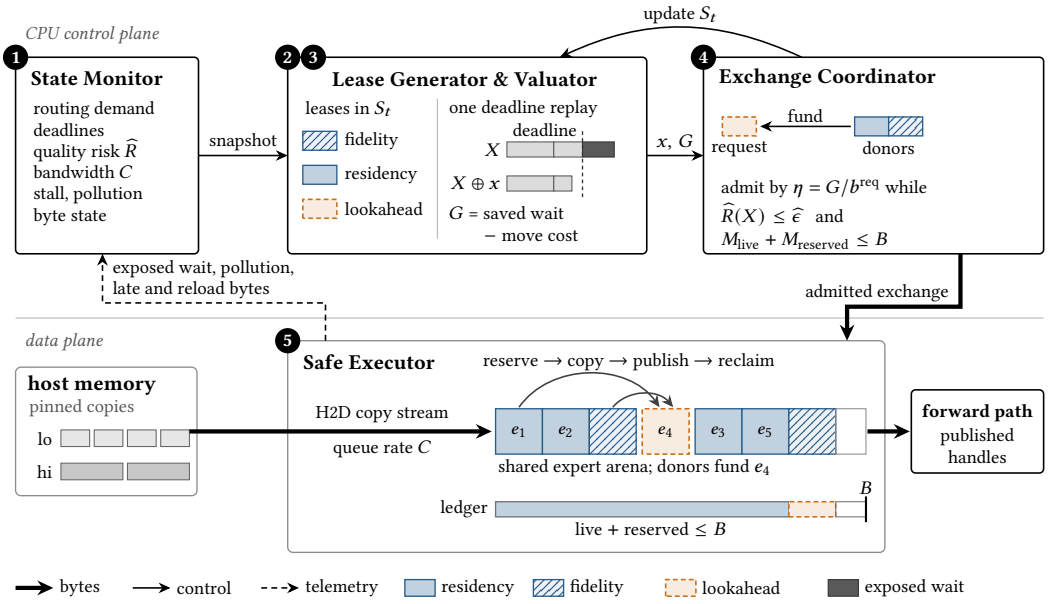


Fig. 5. DynaByte architecture. The State Monitor builds one snapshot (step 1); the Lease Generator emits fidelity, residency, and lookahead requests (step 2); the valuator scores a donor-closed exchange with one deadline replay (step 3); and the Exchange Coordinator admits it while both accounts hold (step 4). The Safe Executor commits the schedule on one host-to-device queue and one expert arena (step 5).

The State Monitor builds one snapshot of routing demand, deadlines, quality risk, bandwidth, and byte state (step 1). The Lease Generator emits fidelity, residency, and lookahead requests inside the candidate envelope S_t (step 2). The coordinator attaches donor leases, and the valuator replays the migration queue before and after the resulting exchange (step 3). The coordinator admits the exchange when predicted exposed wait falls and both the quality-risk and byte accounts hold (step 4). The Safe Executor reserves, copies, publishes, and reclaims according to the admitted schedule (step 5). The return path updates the next snapshot and S_t.

The object passed from step 4 to step 5 is

$$x = (Z^+, D, \mathcal{T}), \tag{1}$$

where Z^+ denotes the requested leases, D the donor leases released to fund them, and $\mathcal{T}$ the transition schedule. The coordinator scores the state after the whole exchange, so the value of one use includes its effect on the other two. Peak expert memory tests whether the transition fits. Quality risk tests the admission proxy.

A runtime policy π chooses the representation, resident set, and transfer issue time of every expert. The budget B is the expert memory remaining after non-expert weights, the KV cache, activations, and fixed workspaces are reserved, and C is the measured host-to-device copy rate defined in Section 2.1. The allocation problem is

$$\min_{\pi} \ \text{P95}(\text{TPOT}; \pi) \quad \text{subject to} \quad M_{\text{peak}}(\pi) \le B, \qquad \Delta Q(\pi) \le \epsilon. \tag{2}$$

DynaByte treats Eq. (2) as that problem. The executor enforces $M_{\text{peak}}(\pi) \le B$ on every transition. The controller admits with $\widehat{R}(X) \le \widehat{\epsilon}$ in place of the $\Delta Q(\pi) \le \epsilon$ test. The deadline replay charges C for the migration-queue time occupied by each exchange. The runtime consults a calibrated risk

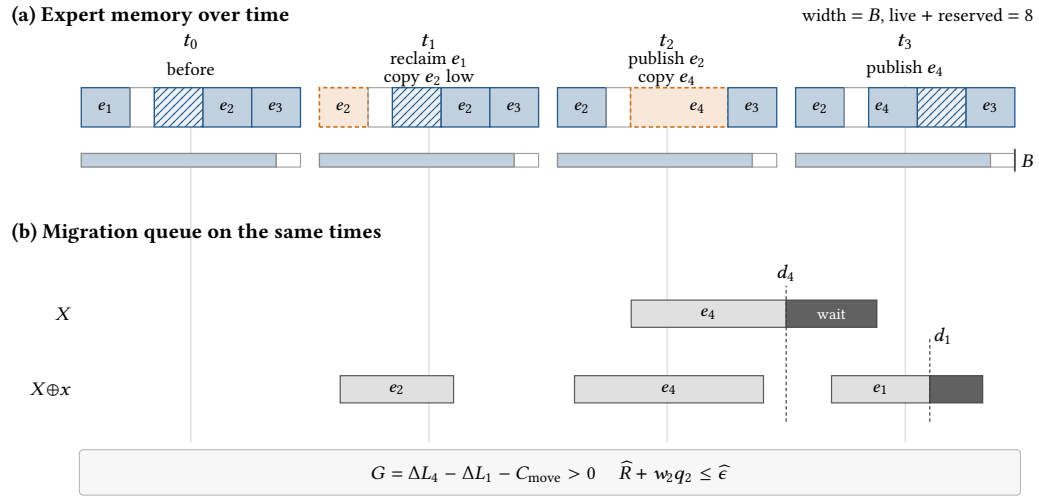


Fig. 6. A donor-backed exchange on one timeline. (a) Every bar has the same width B. Live plus reserved bytes stay at 8 while e_1's residency and e_2's fidelity increment are released to reserve e_4. (b) On that timeline, X misses d_4, and $X \oplus x$ pays for e_2's copy and e_1's reload. The exchange is admitted because $G > 0$ and $\widehat{R} + w_2 q_2 \le \widehat{\epsilon}$.

estimate while making decisions; Section 6.2 explains why this estimate is not a per-request quality guarantee.

An expert is absent, resident at low precision, resident at high precision, or in transition toward a resident state. Its packed footprints are m_i^{lo} and m_i^{hi}. A high-precision resident accounts for a low-precision residency plus the fidelity increment $\delta m_i^{\text{fid}} = m_i^{\text{hi}} - m_i^{\text{lo}}$, while remaining one packed block. Lookahead copies an absent expert before its deadline and becomes residency at the selected tier once the copy is published. The low- and high-precision loads of one absent expert are mutually exclusive. Section 3.5 specifies when a reserved representation may be published and when donor bytes may be reclaimed.

Figure 6 traces one exchange through both budgets. Lookahead e_4 does not fit in the one free unit, and e_1's residency still leaves it short, so the coordinator also releases e_2's fidelity increment. The replay admits the exchange only when e_4's saved wait exceeds the cost of e_1's later reload and e_2's low-precision copy, while the added quality risk remains within the proxy limit.

3.2 State Monitor

The State Monitor performs step 1 in Figure 5: it turns observed runtime facts into one immutable snapshot. It neither predicts future uses nor constructs or admits an exchange. The snapshot contains demand and timing, quality, and execution and byte state.

Demand and timing. The monitor records recent routing transitions, per-layer expert-kernel time, and pre-gating logits when the checkpoint exposes them. Section 3.3 converts these observations into future-use probabilities, deadlines, and reuse estimates.

Quality. An offline sensitivity $q_i \ge 0$ is the increase in calibration loss when only expert i uses its low representation. The runtime does not modify q_i. After each forward step, the monitor updates w_i with an exponential average of expert e's router-weighted dispatch frequency. The product $w_i q_i$ estimates how often the current workload exercises that sensitivity. For a planned state X,

Table 1. How the three uses modify the same memory account, quality account, and migration-queue replay. "Queue effect" is the quantity changed in the replay of Eq. (6).

Use	Bytes	Lifetime	Risk effect	Queue effect
Fidelity	$\delta m_i^{\mathrm{fid}}$	slow period	removes $w_i q_i$	transfer size
Residency	m_i^{lo}	fast period	none by itself	transfer exists
Lookahead	$m_i^{b_i}$	issue to use	selected tier	issue time

$h_i(X) = 1$ when expert i's next execution uses the high representation. The monitor provides

$$\widehat{R}(X) = \sum_i w_i q_i \big(1 - h_i(X)\big). \tag{3}$$

The coordinator admits against $\widehat{R}(X) \le \widehat{\epsilon}$. It does not convert risk into milliseconds.

Execution and byte state. The monitor separates the published state used by dispatch from the reserved bytes of an accepted transition, and reports their locations and byte counts separately. It also records measured bandwidth, exposed wait, late bytes, unused lookahead bytes, and eviction-induced reload bytes. Stall s_t is the fraction of routed-layer time spent waiting for expert transfers. Pollution p_t is unused lookahead bytes plus reload bytes, divided by the lookahead traffic admitted in that period. The monitor reports the exponentially smoothed values $\widetilde{s}_t$ and $\widetilde{p}_t$. Section 3.4 uses them to change S_t; the monitor does not.

The resulting snapshot is the only interface from the monitor to the Lease Generator.

3.3 Lease Generator and Valuator

The Lease Generator performs step 2 in Figure 5: it turns the monitor snapshot into lease requests. The Valuator performs step 3, but only after the coordinator has attached donors and thereby closed a request into an exchange. Neither component admits an exchange.

Bound the candidate set. The adaptive envelope S_t is the maximum routed-layer depth from which the generator may produce requests. It limits candidate generation only and reserves no memory, bandwidth, or lookahead quota. Section 3.4 adjusts S_t from observed stall and pollution. For a resident set R and predicted expert set $\hat{U}(S)$, the generator retains a prefix depth only when

$$\sum_{i \in \hat{U}(S) \setminus R} m_i \le S T_\ell C. \tag{4}$$

This comparison filters candidates. The subsequent replay still uses individual issue times and deadlines.

Forecast uses and form requests. For each retained layer, the forecaster reads pre-gating logits from the snapshot when they are available and otherwise uses recent routing transitions. It produces a predicted use $j = (i_j, b_j, \omega_j, a_j, d_j)$: expert, representation tier, occurrence probability, earliest issue time, and deadline. The deadline is the latest ready time that introduces no transfer wait.

A lease $z = (u, i, b, [t_s, t_f))$ records its use $u \in \{\mathrm{fid}, \mathrm{res}, \mathrm{look}\}$, expert, bytes, and expected lifetime. The interval guides valuation: an early use may shorten a lookahead lease, whereas a late copy retains its bytes until completion or cancellation. Table 1 shows how all three requests enter the same two accounts and migration-queue replay.

A fidelity request claims $\delta m_i^{\mathrm{fid}}$ beyond the base residency lease and lasts for a slow period. A demotion releases that increment only after the low representation is published. A residency request renews m_i^{lo} for one fast period; eviction releases it after outstanding readers finish. For an

absent expert, the generator emits mutually exclusive low- and high-tier lookahead requests when their resulting risk is feasible. A lookahead lease lasts from issue until use, after which it becomes residency at the selected tier or is reclaimed.

Value a closed exchange. The generator sends requests, not scores, to the coordinator. For each request, the coordinator proposes one or more donor sets D and calls the valuator on the resulting exchange $x = (Z^+, D, \mathcal{T})$. The valuator replays transfers in earliest-deadline-first order. A resident expert is ready at the start of the window. An absent expert's completion time includes bytes already queued and its earliest issue time. Uses of the same expert and tier share one readiness event under continuous batching. The predicted exposed wait is

$$\widehat{L}(X) = \sum_j \omega_j \, [r_j(X) - d_j]_+. \tag{5}$$

This scheduling surrogate is calibrated against measured tail latency. It is not a direct estimate of the P95 quantile in Eq. (2).

The valuator returns

$$G(x \mid X) = \widehat{L}(X) - \widehat{L}(X \oplus x) - C_{\text{move}}(x), \tag{6}$$

where $X \oplus x$ applies the request, donors, and transition schedule, and C_{move} is exposed transition cost. The same function scores all three uses: fidelity changes representation and transfer size, residency removes a transfer, and lookahead changes its issue time. One replay prevents a retained expert and its lookahead from both receiving credit for the same avoided transfer, and it charges every released donor for its later reload.

3.4 Exchange Coordinator

The Exchange Coordinator attaches donors before valuation and makes the only admission decision. One control event has three stages: close a request, value and admit the closed exchange, and refresh the next candidate set.

Close. The coordinator initializes provisional state X from the monitor snapshot. Slack, $B - M_{\text{live}} - M_{\text{reserved}}$, funds a request directly when it is sufficient. Otherwise D is built from fidelity leases eligible for demotion, residency leases eligible for eviction, and lookahead leases eligible for cancellation. Reclaimable leases come first: a copy that has not started or is no longer useful, and a resident whose predicted reuse lies beyond S_t. A resident that is still needed can become a donor only when the replay charges its later reload. Donor construction scans leases in increasing counterfactual loss, the extra predicted wait and move cost caused by releasing the lease. In Figure 6, lookahead e_4 is closed with e_1's residency lease and e_2's fidelity increment.

A closed exchange is feasible only when the request and its donors do not conflict, its resulting risk remains within $\widehat{\epsilon}$, and its complete transition satisfies

$$M_{\text{live}}(t) + M_{\text{reserved}}(t) \le B \qquad \forall t. \tag{7}$$

A release becomes byte credit only at its scheduled reclaim time. Two exchanges therefore cannot spend the same donor, and future bytes cannot be spent as current slack.

If the published state exceeds the risk account, the coordinator first restores feasibility through promotion exchanges with the smallest replayed latency cost per unit of risk removed. This repair uses the same closure and valuation path. Once the state is feasible, a demotion can fund another request only when the complete resulting exchange remains feasible.

Value and admit. Let b_x^{req} be the requested lease's bytes over its next lifetime, including bytes renewed by a residency request. The coordinator ranks feasible exchanges with positive G by

$$\eta(x \mid X) = G(x \mid X)/b_x^{\mathrm{req}}. \tag{8}$$

After each admission it updates both provisional accounts, invalidates conflicting requests, and asks the valuator to replay the affected exchanges. Retaining an expert can remove its lookahead request, and changing a representation can alter several completion times. The search stops when no feasible exchange has positive G. Eq. (2) is the offline problem, and Static-Joint solves it once from calibration statistics. Each control event admits the feasible exchange with the largest $\eta = G/b_x^{\mathrm{req}}$, where G is the drop in predicted exposed wait after the donor-closed replay. Because G depends on exchanges already admitted, and Eq. (2) is a tail over requests, η is a snapshot heuristic. Admission still requires $\widehat{R} \le \widehat{\epsilon}$.

Refresh. Every T_{fast} routed layers, the generator refreshes residency and lookahead requests. Every T_{slow} forward steps, it adds fidelity requests to the same pool. Minimum fidelity tenure and replacement hysteresis suppress changes too short to repay their copies. At a fast event, the coordinator increases S_t by one layer when $\widetilde{s}_t - \lambda\widetilde{p}_t > \theta_h$, decreases it by one layer when the difference falls below $-\theta_h$, and leaves it unchanged otherwise. This changes only which future experts the generator may propose. Donor closure, replay, and η still determine admission.

The coordinator's only output is this set of admitted, donor-closed schedules.

3.5 Safe Executor

The Safe Executor performs step 5 in Figure 5. Its input is an admitted schedule $(Z^+, D, \mathcal{T})$. Its output is a new published state and telemetry for the next snapshot. It does not choose among fidelity, residency, and lookahead.

Reserve. The executor reserves every extent in $\mathcal{T}$ in one transaction and checks the transition peak in Eq. (7). A destination that must exist before its donor becomes reclaimable is bound to an actual extent during this transaction. If no compatible extent can be bound, or the peak exceeds B, the executor discards the schedule and leaves the published state unchanged.

Copy and publish. Copies run on the migration stream. A demotion keeps serving the high representation until the low replacement is published. An eviction-only donor is reclaimed after its readers finish. The executor atomically publishes the new handle only after the destination copy completes. Until then the forward path continues to use the old representation.

Reclaim and ready work. The executor reclaims an old extent only after its reader count reaches zero and its recorded uses complete. After routing, groups with published handles issue before groups waiting on a transfer. A missed lookahead waits only in its assigned group, and routing and top-k do not change. A speculative exchange may displace a protected resident only when its donor-closed replay has already charged that resident's reload.

Telemetry. The executor returns the measurements defined in Section 3.2. They update the next snapshot and do not roll back a published representation. This closes the loop from snapshot to request, closed exchange, admitted schedule, and telemetry.

4 IMPLEMENTATION

We implement DynaByte in Python with PyTorch and HuggingFace Transformers [34, 47]. Three mechanisms below are the ones Section 3 leaves to the implementation: the inputs of the forecaster, the bytes stored for each representation tier, and the placement of one extent in the shared arena.

Algorithm 1 One DynaByte control event.

Require: snapshot, published and reserved leases, envelope S_t
if $\widetilde{s}_t - \lambda \widetilde{p}_t > \theta_h$ **then**
  $S_t \leftarrow \min(S_t + 1, S_{\max})$
else if $\widetilde{s}_t - \lambda \widetilde{p}_t < -\theta_h$ **then**
  $S_t \leftarrow \max(S_t - 1, S_{\min})$
end if
$Z \leftarrow$ residency and lookahead requests with depth $\le S_t$
if this event is a slow boundary **then**
  add fidelity requests to Z
end if
if $\widehat{R}(X) > \widehat{\epsilon}$ **then**
  close promotions of minimum replayed cost per unit risk
  admit them until $\widehat{R}(X) \le \widehat{\epsilon}$
end if
for all request $z \in Z$ **do**
  attach donors D in increasing counterfactual loss
  close $x = (z, D, \mathcal{T})$
  if x violates $\widehat{R}$, Eq. (7), or deadlines **then**
    discard x
  else
    compute $G(x \mid X)$ and $\eta(x \mid X)$ by deadline replay
  end if
end for
while some feasible x has $G(x \mid X) > 0$ **do**
  admit $x^\star \leftarrow \arg\max_x \eta(x \mid X)$
  apply $x^\star$ to provisional X and both accounts
  drop conflicting exchanges and replay the rest
end while
submit the admitted schedules $(Z^+, D, \mathcal{T})$

Forecaster. For each layer retained by S_t, the forecaster in Section 3.3 reads pre-gating logits from the snapshot when the snapshot contains them. Otherwise it ranks that layer's experts by recent activation counts and returns a prefix of configured width. Low confidence lengthens this prefix and leaves S_t unchanged. When the forecast is late for the layer that is about to run, the generator uses recent routing history and the forward path continues. An optional CPU model may revise the probabilities after this step. The admission rule reads the forecast from the steps above.

Envelope and representations. S_t is a depth in routed layers. At startup it is the number of layer intervals required to copy one layer of absent-expert bytes at the copy rate C, clipped to the configured bounds. Section 3.4 then updates that depth from observed stall and pollution. Before a timed run, both tiers of each layer are packed into pinned host memory, and that layer's source weights are released once both packs exist. The checkpoint fixes the tier pair, so the two representations stay the same as B changes. While a layer runs, each tier that the layer contains is issued as one grouped GEMM.

Shared arena. Each device-resident expert representation occupies one extent, a contiguous byte range that holds that representation. Layer, tier, and lease purpose share this arena. The arena is backed by one or more preallocated segments, each a contiguous buffer, and every extent is taken from a single segment. The allocator rounds a request up to a multiple of 256 bytes and selects the smallest free range, among all segments, that can hold the rounded size. When a range is returned, adjacent free ranges in the same segment are merged.

5 EVALUATION

We evaluate five questions. RQ1 asks whether DynaByte lowers p95 TPOT at the same expert-memory budget among rows whose measured ΔPPL meets ϵ, and whether that comparison separates from Static-Joint. RQ2 asks when a shared online allocation improves on a sequential order or a frozen assignment, and when one of those simpler policies is enough. RQ3 asks which of counterfactual replay, explicit donors, and separate quotas separate from DynaByte. RQ4 reports the unsafe-accept rate of the quality-risk proxy, the latency-surrogate error, and the controller's runtime cost. RQ5 asks when a simpler policy is sufficient.

5.1 Experimental Setup

Platform and models. We evaluate on a single NVIDIA RTX A6000 GPU with 48 GB of device memory. Table 2 summarizes each checkpoint's routing structure and the two packed tiers of routed experts. For Qwen3-Next, the shared expert is not routed and remains resident with the non-expert weights. We reserve memory for non-expert parameters, the KV cache, activations, conversion workspace, and runtime headroom before allocating the expert-memory budget B.

For Qwen3-30B, this leaves approximately 30 GB for the expert arena. The replay pool occupies 29.74 GB, corresponding to $\Phi = 1.931$. The low- and high-tier expert packs occupy 15.40 GB and 57.98 GB, respectively, giving $\rho = 3.76$; keeping both packs resident would require 73.38 GB and therefore exceeds device capacity. Host-to-device bandwidth in the replay is 24.86 GB/s. Together, Φ and ρ instantiate the two resource constraints introduced in Section 1.

The latency sections use a tighter expert budget, $B = 20.0$ GB ($\Phi = 1.3$). That is the same card with a reserve 9.74 GB larger than the one that leaves 29.74 GB, as when a longer context or a larger batch grows the KV cache and the activations. It is the budget at which a byte spent on fidelity displaces residency or lookahead.

Table 2. Configuration of the evaluated MoE models. High and low tiers are the packed representations of each routed expert.

	Qwen3-30B	Qwen3-Next-80B	Phi-3.5-MoE
Total parameters	30.5B	80B	42B
Activated / token	3.3B	3B	6.6B
High tier	FP16	INT4	FP16
Low tier	INT4	INT2	INT4
Expert weight fraction	95%	96%	96%
Layers	48	48	32
Routed experts / layer	128	512 (+1 shared)	16
Top-K	8	10	2

Workloads and quality. For latency evaluation, we draw length-stratified requests from ShareGPT traces [2], covering prefill-heavy, decode-heavy, and balanced workloads. We time only the model

execution path, excluding request queueing, tokenization, transport, and serialization. We evaluate model quality using WikiText-2 perplexity [33] and five downstream benchmarks spanning knowledge, reasoning, mathematics, and code generation: MMLU-Pro [46], GPQA [38], AIME25 [56], GSM8K [7], and HumanEval [4].

Table 3 compares DynaByte against each model's all-high reference, uniform-low configuration, and Static-Joint. Calibration prompts are used to determine sensitivity estimates, predictor width, controller constants, and baseline configurations; evaluation prompts are disjoint, and no parameter is refit on the test set. We report results for each quality benchmark separately relative to the all-high reference.

The all-high reference executes every routed expert using the model's high-tier representation under the same runtime as DynaByte. Experts not currently resident in HBM are streamed through the same residency path used by the evaluated policies. Thus, all-high defines a fidelity reference rather than an unconstrained placement policy: it does not assume that all high-tier experts can reside in HBM simultaneously.

Baselines. We organize the baselines by the resource-management decision they isolate. *Uniform-low*, *Static-mixed*, *Residency-only*, and *Lookahead-only* isolate individual memory uses while holding the remaining decisions fixed. Seq-FR and Seq-RF test whether independently optimizing the same decisions in a fixed order is sufficient: Seq-FR assigns fidelity before residency and lookahead, whereas Seq-RF assigns residency before fidelity and lookahead. Static-Joint is the offline oracle. On calibration statistics it jointly assigns B among fidelity, residency, and lookahead, keeps the feasible assignment with the lowest latency, and freezes that assignment on the evaluation stream. *Fixed-partition* instead reserves independent quotas for the three uses, testing the value of allowing memory to move across them.

For controlled comparisons, every deployable policy receives the same expert-memory budget B, expert representations, kernels, request stream, and transition interface. Baselines that require calibration receive the same calibration traces and search budget as DynaByte. This keeps the comparison focused on the allocation policy rather than differences in implementation or tuning opportunity.

We additionally compare against HOBBIT in Figure 7. HOBBIT is the published low-precision miss path [45], run here on the same B, representations, kernels, and request stream. We do not run the authors' binary. We do not include DyMoE [21], because our controller does not implement its phase-aware importance estimator, depth-dependent 4/2-bit schedule, or next-layer predictor; approximating these mechanisms would not constitute a faithful reproduction.

Metrics and statistics. We report time to first token (TTFT), time per output token (TPOT), end-to-end latency, and exposed expert-transfer wait. Our confirmatory latency endpoint is the paired difference in request-level p95 TPOT between DynaByte and Static-Joint, evaluated only when the measured quality degradation satisfies $\Delta\text{PPL} \leq \epsilon$. Statistical significance is determined from the paired difference directly; overlap between the two marginal confidence intervals is not used as a significance test.

The SLO column is the percentage of requests whose TPOT is at most Static-Joint's p95 TPOT on that model. On Qwen3-30B that line is 47.2 ms; on Qwen3-Next-80B and Phi-3.5-MoE it is 154 ms and 56.2 ms. Static-Joint scores 96%, 96%, and 95% because a p95 line leaves the rest of its own requests above it. DynaByte scores 97%, 97%, and 96% on those same lines.

Replay measurements use 1,000 measured trials following two warm-up trials on a fixed WikiText routing trace and report exposed transfer wait. End-to-end latency runs use five warm-up iterations followed by 1,000 measured requests for each of five paired seeds. HOBBIT uses the same measurement protocol.

Table 3. WikiText-2 perplexity and downstream scores. High and Low are the curve endpoints. For Qwen3-30B, Static-Joint is 6.99 (ΔPPL 0.58) and DynaByte is 7.03 (ΔPPL 0.62), matching Table 4.

Model	Tier	PPL	MMLU-Pro	GPQA	AIME25	GSM8K	HumanEval	Avg.
Qwen3-30B	High	6.41	76.00	54.04	33.33	90.50	84.76	67.73
	Low	7.19	73.50	50.51	30.00	89.50	84.15	65.53
	Static-Joint	6.99	74.50	53.03	33.33	89.50	84.76	67.02
	DynaByte	7.03	75.00	53.03	33.33	90.00	84.76	67.22
Qwen3-Next-80B	High	5.74	76.00	72.22	70.00	87.00	85.37	78.12
	Low	6.40	71.50	65.66	60.00	86.00	84.76	73.58
	Static-Joint	6.28	75.00	71.21	66.67	87.00	85.37	77.15
	DynaByte	6.26	75.50	71.21	66.67	87.00	85.37	77.15
Phi-3.5-MoE	High	6.41	54.00	36.87	40.00	88.50	70.73	58.02
	Low	7.19	52.00	34.34	36.67	87.50	70.12	56.13
	Static-Joint	7.03	53.00	35.36	40.00	88.50	70.73	57.52
	DynaByte	6.99	53.50	35.86	40.00	88.00	70.73	57.62

Memory accounting and admission. We account for both published and reserved expert-arena bytes and independently sample process-level GPU memory through NVML every 2 ms. A run is considered valid only if arena occupancy remains within the expert-memory budget B and its exchange log can reconstruct the reported allocation. We report process-level memory separately because it additionally includes non-expert weights, KV cache, activations, runtime workspace, and other state outside the expert arena.

5.2 End-to-End Quality–Latency Tradeoff

RQ1: Tail latency at a fixed expert-memory budget. We first ask whether DynaByte reduces tail latency under a fixed expert-memory budget while preserving model quality. We evaluate each model at a binding memory budget and compare policies along the quality–latency frontier. A configuration is considered quality-feasible when its perplexity degradation relative to All-high satisfies $\Delta\text{PPL} \leq \epsilon$. Configurations outside this bound remain visible to expose the latency–quality tradeoff, but are excluded from the feasible latency comparison.

Table 3 reports the quality of the evaluated configurations using both perplexity and downstream task accuracy. For Qwen3-30B, Static-Joint's perplexity is 6.99 (ΔPPL 0.58) and DynaByte's is 7.03 (ΔPPL 0.62), the same degradations as Table 4. We report the downstream tasks separately rather than collapsing them into a single score.

Figure 7 shows the corresponding quality–latency frontier at one binding budget per model: Qwen3-30B at $\Phi = 1.3$ ($B = 20.0$ GB), 9.74 GB below the 29.74 GB arena in Section 5.1, Qwen3-Next-80B at $\Phi = 1.4$, and Phi-3.5-MoE at $\Phi = 1.2$. We set ϵ to 0.70, 0.60, and 0.70, respectively. Each threshold lies between that model's Static-Joint degradation and Uniform-low, so lowering expert fidelity alone can improve latency while missing the quality constraint.

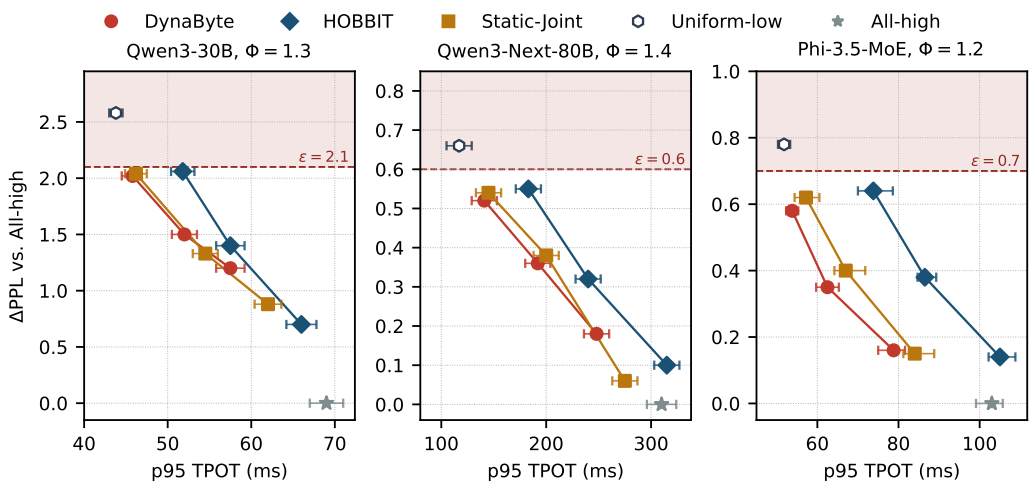


Fig. 7. Quality–latency frontier at a fixed expert-memory budget for each model. Quality is measured as perplexity degradation ΔPPL relative to All-high, and latency as request-level p95 TPOT. The dashed line denotes the model-specific quality constraint ϵ; points above it violate the constraint. Uniform-low achieves low latency but falls outside the feasible quality region.

Table 4. Fastest point of each deployable policy at that model's ϵ (0.70, 0.60, and 0.70).

Model	Policy	ΔPPL	p95 TPOT (ms)	p95 TTFT (s)	Within SJ p95 (%)	Peak (GB)
Qwen3-30B	DynaByte	0.62	45.6 [44.5, 46.9]	3.1	97	38.6
	HOBBIT	0.66	51.8 [50.4, 53.2]	3.8	91	38.6
	Static-Joint	0.58	47.2 [45.9, 48.5]	3.2	96	38.6
	Uniform-low	0.78	43.8 [43.0, 44.6]	2.6	99	34.0
Qwen3-Next-80B	DynaByte	0.52	141 [129, 153]	8.2	97	43.8
	HOBBIT	0.55	183 [171, 195]	10.4	94	43.8
	Static-Joint	0.54	154 [139, 169]	8.6	96	43.8
	Uniform-low	0.66	117 [105, 129]	6.8	99	47.8
Phi-3.5-MoE	DynaByte	0.58	53.8 [52.3, 55.3]	2.2	96	47.1
	HOBBIT	0.64	59.8 [58.3, 61.3]	2.9	74	47.1
	Static-Joint	0.62	56.2 [54.7, 57.7]	2.3	95	47.1
	Uniform-low	0.78	51.8 [50.3, 53.3]	1.8	99	47.1

Table 4 reports the corresponding tail-latency results. Uniform-low attains the lowest p95 TPOT but exceeds the quality threshold on every model, illustrating that latency gains obtained solely by lowering expert fidelity are not sufficient under the quality constraint. We therefore compare DynaByte with the remaining quality-feasible policies using paired request-level measurements. Static-Joint is that offline oracle, so the gap to DynaByte is the value of recomputing the assignment on the evaluation stream. On Qwen3-30B the p95 TPOT pair is 45.6 ms [44.5, 46.9] against 47.2 ms [45.9, 48.5]. On Qwen3-Next-80B it is 141 ms [129, 153] against 154 ms [139, 169]. On Phi-3.5-MoE it is 53.8 ms [52.3, 55.3] against 56.2 ms [54.7, 57.7]. All three pairs of intervals overlap. HOBBIT provides an external point of comparison using token-level precision adaptation. Together,

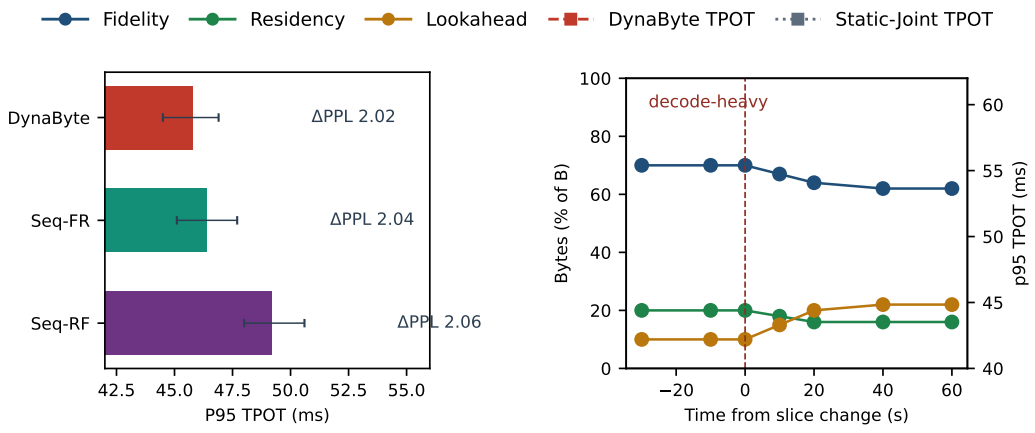
these comparisons test whether dynamically exchanging memory among fidelity, residency, and lookahead improves the feasible quality–latency frontier rather than merely trading quality for lower latency.

5.3 Shared Online Allocation and Simpler Policies

RQ2: When does a shared online allocation improve on a simpler policy? RQ1 evaluates the end-to-end quality–latency tradeoff; we next isolate when sharing the expert-memory budget across competing uses, and adapting that allocation online, is necessary. We compare DynaByte against sequential and static policies under both a stationary workload and a workload shift. The former tests whether joint allocation improves over independently ordered decisions, while the latter tests whether an allocation chosen for one request regime remains effective when the bottleneck changes.

Figure 8(a) evaluates Qwen3-30B at $\Phi = 1.3$ on a balanced-length workload. The bars are DynaByte, Seq-FR, and Seq-RF, annotated at perplexity degradations of 2.02, 2.04, and 2.06. The sequential policies optimize fidelity, residency, and lookahead in a fixed order. This comparison asks whether a shared allocation improves on those fixed orders.

Figure 8(b) then introduces a balanced-to-decode-heavy workload shift. We track both the allocation of the shared expert-memory budget and the resulting p95 TPOT around the transition. The workload shift changes the relative value of memory assigned to fidelity, residency, and lookahead; the offline oracle retains its calibrated partition, whereas DynaByte can reallocate bytes as the active bottleneck changes. This experiment therefore isolates the additional value of *online adaptation* from that of joint allocation alone.



(a) Balanced-length slice

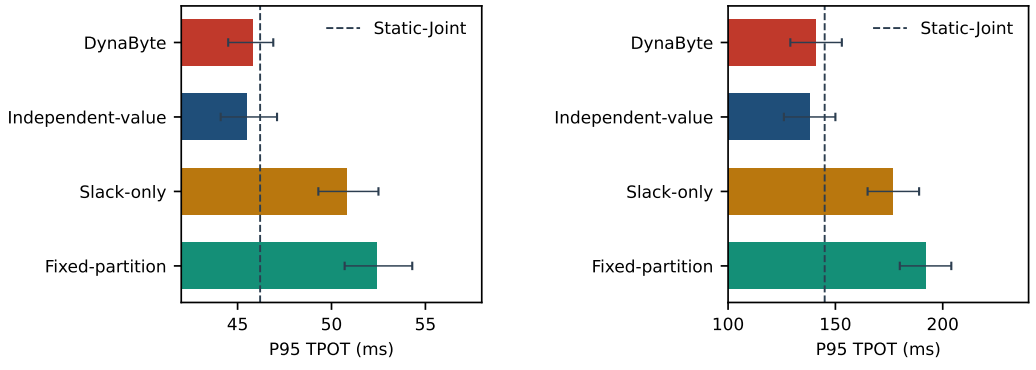
(b) Balanced to decode-heavy

Fig. 8. When shared and online allocation is useful for Qwen3-30B at $\Phi = 1.3$. (a) Under a stationary balanced-length workload, DynaByte is compared with the two sequential orders; labels are perplexity degradation. (b) Under a balanced-to-decode-heavy shift at $t = 0$, byte shares of B and p95 TPOT for DynaByte and Static-Joint.

5.4 Mechanism Attribution

RQ3: Which mechanisms contribute to DynaByte's benefit? We isolate the contribution of DynaByte's key mechanisms by removing one at a time while holding the workload, expert-memory

budget B, and execution path fixed within each experiment. *Independent-value* makes allocation decisions using each candidate's individual value, without accounting for its effect on subsequent exchanges. *Slack-only* restricts admission to currently available slack and disables explicit donor selection. *Fixed-partition*, defined in Section 5.1, replaces the shared arena with separate quotas for fidelity, residency, and lookahead. Together, these ablations test the value of exchange-aware decisions, explicit memory reclamation, and fungible allocation across competing memory uses.



(a) Qwen3-30B, FP16/INT4

(b) Qwen3-Next-80B, INT4/INT2

Fig. 9. Ablation of DynaByte's allocation mechanisms. (a) Qwen3-30B at $\Phi = 1.3$. (b) Qwen3-Next-80B at $\Phi = 1.4$. Each ablation removes one mechanism while keeping the workload, memory budget, and execution path fixed. Horizontal bars denote 95% confidence intervals; the dashed line marks the corresponding Static-Joint latency.

Figure 9 separates three sources of benefit. Comparing *Independent-value* with DynaByte tests whether exchange-aware valuation improves decisions beyond scoring allocations independently. *Slack-only* tests whether opportunistically consuming free space is sufficient, or whether reallocating occupied memory through explicit donors is necessary. *Fixed-partition* tests whether the three memory uses can be provisioned independently or benefit from sharing a fungible budget. For each ablation, we interpret latency only when the resulting configuration also satisfies the model's quality constraint; a lower latency obtained by violating $\Delta\text{PPL} \leq \epsilon$ is not considered a feasible improvement.

5.5 Surrogate Checks and Runtime Cost

RQ4: Are the online surrogates accurate and inexpensive enough for runtime decisions? DynaByte relies on approximate signals to make allocation decisions on the critical path. We validate three such signals—predicted exposed wait, quality risk, and lookahead prediction—and separately measure the cost of evaluating candidate exchanges. Figure 10 reports these checks for Qwen3-30B.

Figure 10(a) compares the latency surrogate $\widehat{L}$ with measured exposed transfer wait. The surrogate estimates the cost of an exchange from the transferred bytes and the execution window available to hide the transfer. On this workload the predictions sit above measured wait, and the panel marks a bias of +1.2 ms per token. Deviations can arise from transfer-bandwidth estimation and from delayed state updates that the surrogate does not represent.

Figure 10(b) evaluates the quality-risk surrogate $\widehat{R}$ against measured ΔPPL. Each point is one admission decision. The vertical line is the predicted-risk limit and the horizontal line is ϵ. The four markers are disjoint: 66 safe accepts, 17 unsafe accepts, 59 rejects, and 24 conservative rejects.

Of the 83 states with $\widehat{R}$ at or below the limit, 17 have measured ΔPPL above ϵ, an unsafe-accept rate of 20.5%. Dividing 17 by the 66 safe accepts gives 26% and leaves the unsafe accepts out of the denominator. Those 17 are not clustered on the boundary: their median measured value is 1.27ϵ, and 11 of them exceed 1.2ϵ. Table 4 is a different object. Each latency row is one complete configuration, and it enters the feasible comparison only when its measured ΔPPL is at or below ϵ. On Qwen3-30B that measured value is 0.62, below $\epsilon = 0.70$. The controller admits on $\widehat{R} \le \widehat{\epsilon}$ and repairs only predicted violations, so those 17 states stay inside the proxy's accepted set. The reported configuration stays within ϵ because that configuration was measured. Conservative rejects are the 24 states the proxy refuses whose measured ΔPPL remains within ϵ.

Figure 10(c) measures lookahead recall at envelope depth three. Recall is 0.71 on prefill-heavy requests, 0.58 on mixed requests, and 0.46 on decode-heavy requests. Recall is the fraction of the subsequently demanded expert set identified early enough to influence allocation.

Finally, Figure 10(d) measures controller cost as the number of exchange candidates K and planning-window size W increase. Our default operating point is $K = 8$ and $W = 32$, where replay time is 0.15 ms, below one layer. The same panel marks a 33–130 ms band once K and W leave that point. We compare controller time with the available execution window to determine whether allocation decisions remain off the critical path, and increase K and W to characterize how the optimization cost scales beyond the default configuration.

Across these panels, the latency surrogate is biased high by 1.2 ms per token, the quality-risk proxy's unsafe-accept rate is 20.5%, lookahead recall falls from 0.71 to 0.46 as requests become decode-heavy, and the default controller stays under one layer at 0.15 ms.

5.6 Robustness and Boundary Regimes

RQ5: When is a simpler policy sufficient? Finally, we ask when online reallocation adds little over the offline oracle. The binding comparisons in Section 5.2 already sit where $1 \le \Phi < \rho$, so fidelity, residency, and lookahead compete for one arena. Figure 11 varies host-to-device bandwidth for Qwen3-30B at $\Phi = 1.3$, with $\epsilon = 2.1$ and batch size 8, under that same budget and workload.

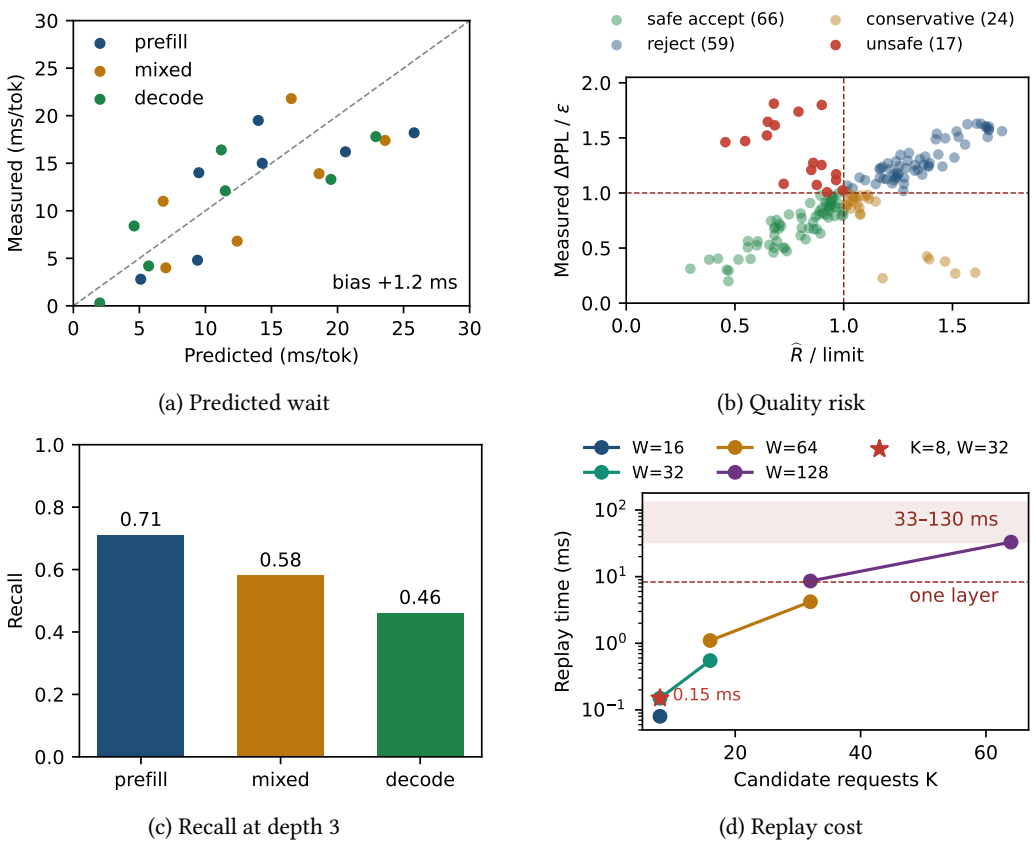
The sweep places two caps below the measured 24.86 GB/s, at 5 and 12 GB/s, the grid point beside that rate at 25 GB/s, and one cap above it at 40 GB/s. The marked point is the measured rate. The four pairs below are the points printed on Figure 11.

At 5 GB/s, p95 TPOT is 536 ms for DynaByte and 608 ms for Static-Joint. At 12 GB/s the pair is 498 ms against 540 ms. At the marked native rate the pair is 458 ms against 462 ms, and at 40 GB/s it is 446 ms against 450 ms. The gap is 72 ms at 5 GB/s and 42 ms at 12 GB/s, then 4 ms at 25 GB/s and at 40 GB/s. On this grid the gap falls from 42 ms to 4 ms between 12 GB/s and 25 GB/s, a span of 13 GB/s. The native rate, 24.86 GB/s, lies in the 4 ms pair, 12.9 GB/s above the last point that still shows 42 ms.

6 DISCUSSION

6.1 Applicability and Boundary Regimes

Joint allocation matters only when the expert-memory budget leaves a meaningful choice among fidelity, residency, and lookahead. When $\Phi \ge \rho$, every expert can remain resident at high precision and the conflict disappears. The measurements in Section 5 use budgets below that point. For Qwen3-30B, $\rho = 3.76$ and the replay pool in Section 5.1 has $\Phi = 1.931$, so a high-precision copy of every expert still does not fit. At this leftover arena the low-tier pack fits, and exposed transfer wait at the native copy rate is zero. When $1 \le \Phi < \rho$, the aggregate low-tier footprint fits, but every fidelity increment consumes bytes that could otherwise retain or stage an expert. The three uses compete in this band. Table 4 uses the tighter budget $\Phi = 1.3$, 9.74 GB below that arena. At the



(a) Predicted wait

(b) Quality risk

(c) Recall at depth 3

(d) Replay cost

Fig. 10. Validation of the online decision signals and controller cost for Qwen3-30B. (a) Predicted versus measured exposed transfer wait; the diagonal denotes perfect prediction. (b) Predicted quality risk versus measured ΔPPL. The four markers are disjoint. Safe accepts and unsafe accepts are the states with $\widehat{R}$ at or below the limit; the unsafe-accept rate divides the unsafe count by that sum. (c) Lookahead recall at envelope depth three. (d) Controller cost as candidate count K and planning-window size W increase; the star marks the default $K = 8$, $W = 32$ configuration.

native rate, Qwen3-30B p95 TPOT is 45.6 ms [44.5, 46.9] for DynaByte and 47.2 ms [45.9, 48.5] for the offline oracle. Qwen3-Next-80B is 141 ms [129, 153] against 154 ms [139, 169], and Phi-3.5-MoE is 53.8 ms [52.3, 55.3] against 56.2 ms [54.7, 57.7]. On each model the two intervals overlap. When $\Phi < 1$, some experts must remain absent even at low precision.

Host-to-device bandwidth is the other boundary. Figure 11 reports it for Qwen3-30B at $\Phi = 1.3$: p95 TPOT is 536 ms for DynaByte and 608 ms for Static-Joint at 5 GB/s, and 458 ms against 462 ms at the marked native rate. At 40 GB/s the pair is 446 ms against 450 ms. Section 5.6 measures the width of that gap: it falls from 42 ms at 12 GB/s to 4 ms at 25 GB/s, between those two grid points. Table 4 is the end-to-end native-rate comparison, and on each model those intervals overlap. The printed separation from the offline oracle is at the 5 and 12 GB/s caps. On a stationary trace the offline oracle already places fidelity and residency on the calibration hot set. When the workload shifts from balanced to decode-heavy, Figure 8(b) keeps that oracle at its calibrated shares while DynaByte moves bytes among the three uses. Joint allocation changes latency when $1 \le \Phi < \rho$,

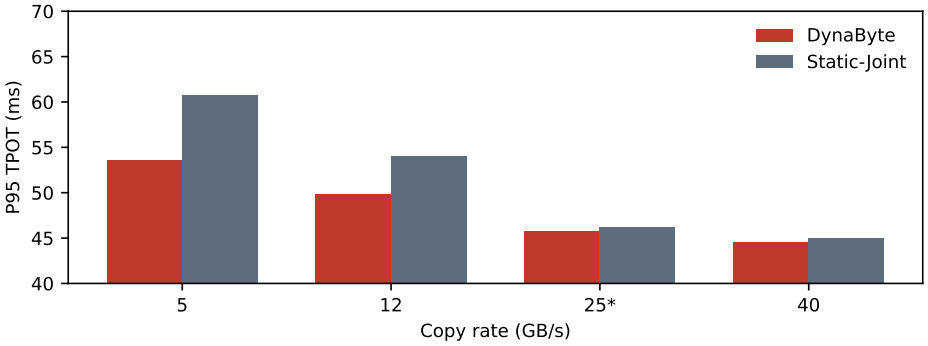


Fig. 11. Sensitivity to host-to-device bandwidth for Qwen3-30B at $\Phi = 1.3$, $\epsilon = 2.1$, and batch size 8. We compare p95 TPOT for DynaByte and Static-Joint under the same expert-memory budget and workload. The marked operating point denotes the native measured bandwidth. The rates are two caps below the measured 24.86 GB/s (5 and 12), the neighboring grid point (25), and one cap above it (40).

and the online policy separates from this oracle when copy time exceeds the overlap window or the routed set leaves the calibration hot set.

6.2 Guarantees and Estimation Limits

The design separates properties enforced by the executor from quantities estimated by the controller. The quality account combines offline sensitivity q_i, which estimates the consequence of using expert i's low representation, with routing weight w_i, which estimates how often the current workload exercises that consequence. Their product is useful for admission, but it need not be additive across experts or stable across tasks. Interactions among quantized experts can invalidate a sum of individual sensitivities, and workload shift can invalidate the calibration values. The runtime therefore treats $\widehat{R}(X) \le \widehat{\epsilon}$ as a feasibility proxy and never converts risk into milliseconds. The actual constraint $\Delta Q \le \epsilon$ is evaluated on complete configurations, with $\widehat{\epsilon}$ fixed before the evaluation workload is observed. In Figure 10(b), 17 of the 83 states the proxy accepts have measured ΔPPL above ϵ, an unsafe-accept rate of 20.5%. The executor enforces the expert-memory peak, while each feasible row in Table 4 is one complete configuration checked after measurement. On Qwen3-30B that measured degradation is 0.62 against $\epsilon = 0.70$. Online admission is a filter with that miss rate, and the end-to-end quality statement is the measured configuration.

Memory safety and routing semantics do not depend on these estimates. Latency efficiency and quality-risk admission do. Section 5.5 shows that predicted wait sits above measured wait. A speculative exchange prices a reload only inside the current envelope, so reuse beyond that envelope can still be missed.

6.3 System Scope and Deployment Assumptions

The current runtime manages one accelerator and uses host memory as its backing store. Host memory retains both packed representations for the process lifetime; a deployment that cannot do so would need to quantize on demand or read a representation from storage, adding a transition not modeled here. Resident-first execution assumes that expert groups within a layer can be issued independently. A kernel that issues experts in a fixed order needs a different latency estimate.

Expert-parallel execution and an SSD tier are outside the present design. They add per-device capacity, network or storage queues, and load imbalance, so the replay and the ledger would have to include those queues.

6.4 Validity, Reproducibility, and Ethics

The motivating measurements and trace replay in Sections 2.2–2.5 use configurations that differ from the joint runtime. They establish quality sensitivity, routing variation, and transfer pressure, not an end-to-end performance result for DynaByte. Claims based on their absolute values must be repeated under the final allocator, exchange policy, and memory accounting.

ShareGPT slices do not cover long contexts, multimodal requests, or rapidly changing request mixtures, any of which can change routing breadth, the KV reserve, and available overlap. Quality results stay per task, because the benchmarks sample different capabilities and can be too small to resolve a modest change. A device name is not a cause: the policy uses the measured bandwidth and kernel time, while kernels, runtime stacks, transfer paths, and quantization support can still differ across devices.

A result is reproducible under Eq. (2) only when the logs suffice to verify B, ϵ, and the executed state transitions.

The study uses public latency traces and quality benchmarks, recruits no participants, collects no new user data, and sends no prompts to a third-party service. Released artifacts will identify source prompts without redistributing material whose license forbids it.

7 RELATED WORK

Prior systems control expert precision, residency, or transfer timing, often while holding the other decisions fixed. We group them by that control decision, then compare DynaByte directly with systems that already combine the mechanisms.

Precision assignment. GPTQ, AWQ, SmoothQuant, LLM.int8(), OmniQuant, and SqueezeLLM fix representations before serving [10, 18, 26, 32, 39, 48]. ZeroQuant and AQLM are further offline compressors for transformer weights [14, 52]. QMoE compresses MoE weights below one bit per parameter [17]. MoE-specific methods use expert sensitivity, activation frequency, or router statistics to assign precision under a memory or quality target [5, 6, 13, 27, 55]. Such methods improve fidelity for a chosen footprint but commonly evaluate the assignment under fixed placement and transfer policies. DynaByte can use their packed representations and offline sensitivities; its additional question is whether a fidelity increment is worth the residency or lookahead displaced at runtime. Mixed-precision kernels such as MxMoE remain complementary [13]: they determine the compute cost of a selected representation, not which state should occupy expert memory.

Residency and placement. Expert caches retain recently or frequently used experts and select victims using routing history, layer position, or co-activation [3, 28, 31, 49, 53, 58]. These caches primarily estimate residency value with a fixed expert representation. Fiddler evaluates experts that are absent from GPU memory on the CPU, and SiDA builds a data-aware plan for serving large MoE models in limited accelerator memory [12, 25]. PowerInfer keeps frequently activated neurons of sparse feed-forward layers on the GPU and evaluates the remaining neurons on the CPU [44]. Placement methods colocate co-activated experts or map them according to topology and load [8, 51]; their multi-device communication objective is outside our single-accelerator implementation. DeepSpeed-MoE and Tutel run expert-parallel training and inference across devices [22, 36]. MegaBlocks supplies grouped sparse kernels for MoE training [19]. In DynaByte, a residency lease is valued together with the fidelity or lookahead request that would displace it, so eviction cost belongs to the complete exchange rather than to an independent cache policy.

Prefetch and mixed-precision offloading. Cross-layer prefetchers predict future experts and overlap transfers with current computation, using either fixed lookahead or adaptation from observed demand and system state [15, 23, 43]. ExpertFlow coordinates expert scheduling and movement under offload [41]; a separate work with the same name changes token assignment to improve activation efficiency [20]. Their principal control variable is transfer timing for a given expert size and quality policy.

HOBBIT combines dynamic mixed-precision loading, adaptive prefetching, and a multi-level cache; its low-precision miss path reduces transfer cost for experts judged less critical [45]. DyMoE combines online expert importance, depth-aware mixed precision, caching, and lookahead prefetching, and some configurations omit selected experts [21]. These systems establish that the three mechanisms can coexist. The distinction is whether one exchange decides both which bytes occupy expert memory and which transfers occupy the migration queue. Measured quality is reported separately from the risk target used at admission. Section 5.1 includes HOBBIT under the same budget, representations, kernels, and request stream; DyMoE is not included, because the controller does not implement its estimator, depth-dependent bit schedule, or next-layer predictor.

Serving and memory management. Orca, vLLM, and SGLang schedule requests and manage KV-cache memory [29, 54, 57]. Sarathi-Serve chunks long prefills so decode iterations continue, and DistServe and Splitwise place prefill and decode on separate resources [1, 35, 59]. ZeRO-Infinity offloads training state across GPU, CPU, and NVMe, and FlexGen schedules dense-model inference across GPU, CPU, and disk [37, 42]. DynaByte operates on the expert-memory budget left after non-expert state is reserved and is complementary to those serving functions. Because a serving engine can outgrow its declared KV or activation reserve, we report the runtime ledger and process-level peak separately.

8 CONCLUSION

DynaByte allocates expert memory and host-to-device bandwidth among fidelity, residency, and lookahead on one byte ledger. One byte ledger and a publish-after-copy protocol keep the expert-memory peak within budget during a transition. In Figure 9, a separate quota is slower than DynaByte. The gain over the offline oracle is unresolved in Table 4, and at the marked copy rate in Figure 11 the two p95 TPOT values differ by 4 ms.

REFERENCES

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*. 117–134. https://www.usenix.org/conference/osdi24/presentation/agrawal

[2] Anonymous Contributors. 2023. ShareGPT Vicuna Unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. Hugging Face dataset.

[3] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Vol. 1. 715–730. https://doi.org/10.1145/3669940.3707267

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374

[5] Krishna Teja Chitty-Venkata, Jie Ye, and Murali Emani. 2025. MoPEQ: Mixture of Mixed Precision Quantized Experts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*. https://arxiv.org/abs/2509.02512 Bivision Workshop.

[6] Mohammed Nowaz Rabbani Chowdhury, Kaoutar El Maghraoui, Hsinyu Tsai, Naigang Wang, Geoffrey W. Burr, Liu Liu, and Meng Wang. 2026. Efficient Quantization of Mixture-of-Experts with Theoretical Generalization Guarantees. In *The Fourteenth International Conference on Learning Representations*. https://arxiv.org/abs/2604.06515

[7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] https://arxiv.org/abs/2110.14168

[8] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv:2401.06066 [cs.CL] https://arxiv.org/abs/2401.06066

[9] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, et al. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434 [cs.CL] https://arxiv.org/abs/2405.04434

[10] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, Vol. 35. 30318–30332. https://doi.org/10.52202/068431-2198

[11] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, et al. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 5547–5569. https://proceedings.mlr.press/v162/du22c.html

[12] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai H. Li, and Yiran Chen. 2024. SiDA: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models. In *Proceedings of Machine Learning and Systems*, Vol. 6. 224–238. https://proceedings.mlsys.org/paper_files/paper/2024/hash/698cfaf72a208aef2e78bcac55b74328-Abstract-Conference.html

[13] Haojie Duanmu, Xiuhong Li, Zhihang Yuan, Size Zheng, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. 2025. MxMoE: Mixed-precision Quantization for MoE with Accuracy and Performance Co-Design. arXiv:2505.05799 [cs.LG] https://arxiv.org/abs/2505.05799

[14] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme Compression of Large Language Models via Additive Quantization. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 12284–12303. https://proceedings.mlr.press/v235/egiazarian24a.html

[15] Artyom Eliseev and Denis Mazur. 2023. Fast Inference of Mixture-of-Experts Language Models with Offloading. arXiv:2312.17238 [cs.LG] https://arxiv.org/abs/2312.17238

[16] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. https://jmlr.org/papers/v23/21-0998.html

[17] Elias Frantar and Dan Alistarh. 2023. QMoE: Practical Sub-1-Bit Compression of Trillion-Parameter Models. arXiv:2310.16795 [cs.LG] https://arxiv.org/abs/2310.16795

[18] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.17323

[19] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. In *Proceedings of Machine Learning and Systems*, Vol. 5. 288–304.

[20] Xin He, Shunkang Zhang, Kaijie Tang, Shaohuai Shi, Yuxin Wang, Zihao Zeng, Zhenheng Tang, Xiaowen Chu, Haiyan Yin, Ivor W. Tsang, and Yew Soon Ong. 2024. ExpertFlow: Efficient Mixture-of-Experts Inference via Predictive Expert Caching and Token Scheduling. arXiv:2410.17954 [cs.LG] https://arxiv.org/abs/2410.17954

[21] Yuegui Huang, Zhiyuan Fang, Weiqi Luo, Ruoyu Wu, Wuhui Chen, and Zibin Zheng. 2026. DyMoE: Dynamic Expert Orchestration with Mixed-Precision Quantization for Efficient MoE Inference on Edge. arXiv:2603.19172 [cs.LG] https://arxiv.org/abs/2603.19172

[22] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, HoYuen Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. 2023. Tutel: Adaptive Mixture-of-Experts at Scale. In *Proceedings of Machine Learning and Systems*, Vol. 5. 269–287.

[23] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 1018–1031. https://doi.org/10.1109/ISCA59077.2024.00078

[24] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG] https://arxiv.org/abs/2401.04088

[25] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. In *The Thirteenth International Conference on Learning Representations*. https://arxiv.org/abs/2402.07033

[26] Sehoon Kim, Coleman Richard Charles Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. 2024. SqueezeLLM: Dense-and-Sparse Quantization. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 23901–23923. https://proceedings.mlr.press/v235/kim24f.html

[27] Young Jin Kim, Raffy Fahim, and Hany Hassan Awadalla. 2023. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. arXiv:2310.02410 [cs.LG] https://arxiv.org/abs/2310.02410

[28] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6710–6720. https://doi.org/10.18653/v1/2024.acl-long.363

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626. https://doi.org/10.1145/3600006.3613165

[30] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*.

[31] Jiamin Li, Qiang Su, Yitao Yang, Yimin Jiang, Cong Wang, and Hong Xu. 2023. Adaptive Gating in Mixture-of-Experts based Language Models. arXiv:2310.07188 [cs.CL] https://arxiv.org/abs/2310.07188

[32] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, Vol. 6. 87–100. https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[33] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations*. https://arxiv.org/abs/1609.07843

[34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. 8026–8037. https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[35] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. https://doi.org/10.1109/ISCA59077.2024.00019

[36] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 18332–18346. https://proceedings.mlr.press/v162/rajbhandari22a.html

[37] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. https://doi.org/10.1145/3458817.3476205

[38] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2024. GPQA: A graduate-level Google-proof Q&A benchmark. In *First Conference on Language Modeling*. https://arxiv.org/abs/2311.12022

[39] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2024. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *International Conference on Learning Representations*.

[40] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*.

[41] Zixu Shen, Kexin Chu, Yifan Zhang, Dawei Xiang, Runxin Wu, and Wei Zhang. 2025. ExpertFlow: Adaptive Expert Scheduling and Memory Coordination for Efficient MoE Inference. arXiv:2510.26730 [cs.PF] https://arxiv.org/abs/2510.26730

[42] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. 31094–31116. https://proceedings.mlr.press/v202/sheng23a.html

[43] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-based LLM Serving using Proactive Caching. arXiv:2410.22134 [cs.LG] https://arxiv.org/abs/2410.22134

[44] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606. https://doi.org/10.1145/3694715.3695964

[45] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference. arXiv:2411.01433 [cs.LG] https://arxiv.org/abs/2411.01433

[46] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. 2024. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems*, Vol. 37. 95266–95290. https://doi.org/10.52202/079017-3018

[47] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45. https://aclanthology.org/2020.emnlp-demos.6/

[48] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*. 38087–38099. https://proceedings.mlr.press/v202/xiao23c.html

[49] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. arXiv:2401.14361 [cs.LG] https://arxiv.org/abs/2401.14361

[50] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[51] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhabaleswar K. Panda. 2024. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 915–925. https://doi.org/10.1109/IPDPS57955.2024.00086

[52] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers. In *Advances in Neural Information Processing Systems*, Vol. 35. 27168–27183. https://doi.org/10.52202/068431-1970

[53] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2025. EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices. *IEEE Transactions on Mobile Computing* 24, 8 (2025), 7059–7073. https://doi.org/10.1109/TMC.2025.3546466

[54] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*. 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[55] Jinhao Zhang, Yunquan Zhang, Boyang Zhang, Zeyu Liu, and Daning Cheng. 2025. MoQE: Improve Quantization Model performance via Mixture of Quantization Experts. arXiv:2508.09204 [cs.LG] https://arxiv.org/abs/2508.09204

[56] Yifan Zhang and Math-AI Team. 2025. American Invitational Mathematics Examination (AIME) 2025. Hugging Face dataset. https://huggingface.co/datasets/math-ai/aime25

[57] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37. 62557–62583. https://doi.org/10.52202/079017-2000

[58] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9. https://doi.org/10.1145/3676536.3676741

[59] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*. 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[60] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *Advances in Neural Information Processing Systems*, Vol. 35. 7103–7114. https://doi.org/10.52202/068431-0515